□ Experiment 3: Multiple Branching & Conflict Demonstration

🎯 Aim

To create multiple branches, perform operations, demonstrate conflicts, and compare changes between branches.

⚙□ Step 1: Initialize Repository

```
mkdir DataScienceProject3
cd DataScienceProject3
git init
```

🌐 Step 2: Connect to GitHub

☞ Create repository on GitHub (name: DataScienceProject3)

```
git remote add origin https://github.com/your-username/DataScienceProject3.git
git remote -v
```

🌿 Step 3: Create Branch GIT-EXAM-1

```
git checkout -b GIT-EXAM-1
```

🐍 Step 4: Create program2.py (Palindrome Check)

```
touch program2.py
notepad program2.py
```

☞ Code:

```
str1 = input("Enter a string: ")

if str1 == str1[::-1]:
    print("Palindrome")
else:
    print("Not Palindrome")
```

🐍 Step 5: Create program1.py (Squares 1 to 10)

```
touch program1.py
notepad program1.py
```

☞ Code:

```
for i in range(1, 11):
    print(i*i)
```

💾 Step 6: Commit & Push (GIT-EXAM-1)

```
git add .
git commit -m "Added palindrome and squares program"
git push origin GIT-EXAM-1
```

🌿 Step 7: Create New Branch GIT-EXAM-2

```
git checkout -b GIT-EXAM-2
```

🔄 Step 8: Pull Changes from GIT-EXAM-1

```
git pull origin GIT-EXAM-1
```

✍ Step 9: Create program2.py (Again Palindrome)

```
notepad program2.py
```

👉 Modify or rewrite code (to show difference)

```
s = input("Enter string: ")

rev = "".join(reversed(s))

if s == rev:
    print("Palindrome")
else:
    print("Not Palindrome")
```

⚠️ Step 10: Modify program1.py (To Create Conflict)

```
notepad program1.py
```

👉 Change code:

```
for i in range(1, 11):
    print("Square of", i, "is", i*i)
```

💾 Step 11: Commit & Push (GIT-EXAM-2)

```
git add .
git commit -m "Modified files to demonstrate conflict"
git push origin GIT-EXAM-2
```

🔍 Step 12: Show Differences Between Branches

```
git diff GIT-EXAM-1 GIT-EXAM-2
```

👉 OR:

```
git log --oneline --graph --all
```

🔀 Step 13: Create Pull Request (GIT-EXAM-2 → GIT-EXAM-1)

👉 On GitHub:

1. Open repository
2. Click **Compare & Pull Request**
3. Base → `GIT-EXAM-1`
4. Compare → `GIT-EXAM-2`
5. Click **Create Pull Request**

⚠️ Step 14: Resolve Merge Conflict (Important)

👉 If conflict occurs:

- Git will show conflict markers:

```
<<<<<<< HEAD
(code from GIT-EXAM-1)
=======
(code from GIT-EXAM-2)
>>>>>>> branch
```

👉 Fix manually → save file

Then run:

```
git add .
git commit -m "Resolved merge conflict"
git push origin GIT-EXAM-2
```

□ Experiment 4: Git Ignore (.gitignore) Implementation
🎯 Aim

To initialize a Git repository, push it to GitHub, create files, and use `.gitignore` to ignore specific files and directories.

⚙□ Step 1: Initialize Repository

```
mkdir DataScienceProject4
cd DataScienceProject4
git init
```

🌐 Step 2: Connect to GitHub

👉 Create repository on GitHub (name: DataScienceProject4)

```
git remote add origin https://github.com/your-username/DataScienceProject4.git
git remote -v
```

🌿 Step 3: Create Branch GIT-EXAM-1

```
git checkout -b GIT-EXAM-1
```

🐍 Step 4: Create program2.py (Palindrome Program)

```
touch program2.py
notepad program2.py
```

👉 Code:

```
str1 = input("Enter a string: ")
```

```
if str1 == str1[::-1]:
    print("Palindrome")
else:
    print("Not Palindrome")
```

🪢 Step 5: Create program1.py (Squares 1–10)

```
touch program1.py
notepad program1.py
```

👉 Code:

```
for i in range(1, 11):
    print(i*i)
```

💾 Step 6: Commit & Push

```
git add .
git commit -m "Added palindrome and squares program"
git push origin GIT-EXAM-1
```

📁 Step 7: Create IGNORE Directory & Files

```
mkdir IGNORE
cd IGNORE
touch test1.txt test2.txt
cd ..
```

👉 These files should NOT be pushed to GitHub.

🚫 Step 8: Create .gitignore File

```
touch .gitignore
notepad .gitignore
```

👉 Add this line:

```
IGNORE/
```

👉 This tells Git to ignore the entire folder.

💾 Step 9: Commit & Push .gitignore

```
git add .gitignore
git commit -m "Added .gitignore to ignore IGNORE folder"
git push origin GIT-EXAM-1
```

🔍 Step 10: Verify Ignored Files

👉 Run:

```
git status
```